Clinical Workflow Intelligence Platform

Complete System Design and Prototype Architecture

Purpose: A resume-aligned healthcare software prototype that supports clinical information review without duplicating brain-imaging analysis or patient-treatment products.

Project positioning: The platform organizes clinical documents, retrieves relevant information, generates grounded summaries and answers, supports human review, and exports approved information.

Scope: This is a supporting clinical-information workflow application. It does not diagnose diseases, analyze MRI/fMRI images, generate brain maps, recommend treatment, or replace a clinician.

1. Executive Summary

The Clinical Workflow Intelligence Platform is a proposed application for authorized healthcare users. It connects structured patient metadata with searchable clinical documents. Users can upload reports, retrieve relevant sections, ask grounded questions, review AI-generated responses, and export approved information.

Core idea: Clinical information is uploaded, processed, indexed, retrieved through semantic search, and presented with source references so the user can verify the answer against the original document.

2. Problem Statement

• Clinical information may be distributed across patient records, previous reports, doctor notes, and uploaded documents.

• Healthcare staff may spend time searching documents manually for relevant historical information.

• General-purpose AI can produce unsupported answers if it is not grounded in trusted source material.

• A lightweight system is needed to connect structured patient data with searchable clinical documents.

3. Proposed Solution

The platform combines a Flask backend, SQL database, document-processing pipeline, FAISS vector search, and Google Gemini. The application retrieves relevant document sections before generating an answer. Each response includes source references so the user can verify the information.

4. Project Boundaries

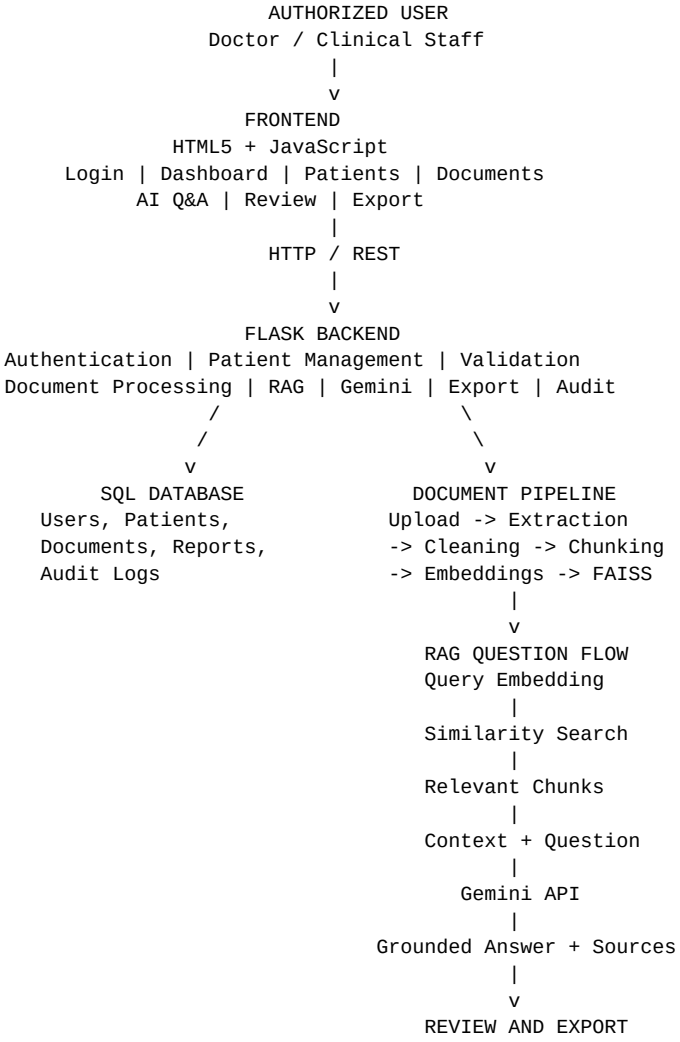
Included	Not included
Clinical document upload and storage	MRI/fMRI image analysis
Patient and document metadata	Brain mapping or connectomics
Semantic search over uploaded documents	Disease diagnosis
Grounded AI summaries and Q&A;	Autonomous medical decisions
Review, approval, and export	Replacement for a clinician

5. Resume Skill Mapping

Resume skill	Project usage
Python	Backend logic, processing, validation, and AI pipeline
Flask	REST APIs and server-side application
HTML5 / JavaScript	User interface and interactive screens
Google Gemini	Grounded summaries and question answering
RAG	Retrieval of relevant document context before generation
SQL	Users, patients, documents, reports, and audit records
FAISS	Vector similarity search
NumPy / Pandas	Data transformation and processing
Testing / Debugging	API validation and reliability checks
Git / GitHub	Version control and collaboration

Resume skill	Project usage
Hugging Face	Optional deployment or demonstration hosting

6. High-Level System Architecture



Architecture Explanation

The frontend sends requests to Flask through REST APIs. Flask authenticates the user, validates inputs, coordinates database operations, and invokes document-processing or AI services. SQL stores structured application data, while the RAG pipeline converts documents into searchable vectors. During question answering, FAISS retrieves relevant chunks and Gemini generates a grounded answer from that context.

7. Main Modules

• **Authentication and authorization:** Controls access to the application and patient records.

• **Patient management:** Creates and manages patient metadata and links documents to a patient.

• **Document management:** Uploads, validates, stores, and lists clinical documents.

• **Document processing:** Extracts text, cleans it, and divides it into searchable chunks.

• **Vector indexing:** Creates embeddings and stores them in FAISS.

• **Question answering:** Retrieves relevant chunks and sends grounded context to Gemini.

• **Review and approval:** Allows users to review and edit AI output before approval.

• **Export:** Generates a structured PDF or downloadable report.

• **Audit logging:** Records important actions such as upload, search, review, and export.

8. End-to-End Workflow

```
1. User logs in.
2. Backend verifies credentials and permissions.
3. User creates or selects a patient.
4. User uploads a clinical document.
5. Backend validates file type, size, ownership, and metadata.
6. Document is stored and marked for processing.
7. Python extracts and cleans text.
8. Text is divided into overlapping chunks.
9. Embeddings are generated for each chunk.
10. Vectors are indexed in FAISS with source metadata.
11. User asks a question about the patient's documents.
12. Question is embedded and compared with FAISS vectors.
13. Relevant chunks are retrieved.
14. Gemini receives the question, context, and safety rules.
15. Answer and source references are returned.
16. User reviews, edits, and approves the report.
17. Approved information can be exported.
```

9. RAG Pipeline

Retrieval-Augmented Generation reduces unsupported answers by retrieving relevant source content before calling the language model.

Stage	Description
Ingestion	Receive and validate uploaded documents.
Extraction	Convert document content into plain text.
Chunking	Split text into manageable sections with overlap.
Embedding	Convert each section into a numerical vector.
Indexing	Store vectors in FAISS and retain source metadata.
Query embedding	Convert the user question into a vector.
Similarity search	Retrieve the most relevant document chunks.
Prompt construction	Combine question, context, and response rules.
Generation	Ask Gemini to answer only from the supplied context.
Citation mapping	Attach source document and section references.

RAG Question-Answering Flow

```
User Question
      |
      v
Query Embedding
      |
      v
FAISS Similarity Search
      |
      v
Top Relevant Document Chunks
      |
      v
Prompt = Question + Retrieved Context + Safety Rules
      |
      v
Gemini API
      |
```

```
      v
Grounded Answer + Source References
      |
      v
Frontend Display -> Human Review -> Approval -> Export
```

10. Database Design

The SQL database stores structured information. The actual vector similarity search is handled by FAISS, while the database stores the metadata needed to connect vectors and chunks back to patients and documents.

Table	Important fields
users	id, name, email, password_hash, role, created_at
patients	id, patient_code, display_name, date_of_birth, created_at
documents	id, patient_id, filename, file_path, document_type, status, uploaded_by, created_at
document_chunks	id, document_id, chunk_index, text, vector_reference
reports	id, patient_id, question, answer, sources, status, reviewed_by, created_at
audit_logs	id, user_id, action, resource_type, resource_id, timestamp

Database Relationships

```
User
  |
  | uploads
  v
Document ---- belongs to ----> Patient
  |
  | divided into
  v
Document Chunks ---- indexed in ----> FAISS Vector Store

Patient
  |
  | has
  v
Reports

User
  |
  | performs actions recorded in
  v
Audit Logs
```

11. REST API Design

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate user
GET	/api/patients	List accessible patients
POST	/api/patients	Create patient record
GET	/api/patients/{id}	Get patient details
POST	/api/patients/{id}/documents	Upload document
GET	/api/patients/{id}/documents	List patient documents
POST	/api/documents/{id}/process	Process and index document
POST	/api/patients/{id}/ask	Ask grounded question
POST	/api/reports/{id}/approve	Approve reviewed report

Method	Endpoint	Purpose
GET	/api/reports/{id}/export	Export report

12. Frontend Screens

• Login screen

• Dashboard with patient and document counts

• Patient list with search and filters

• Patient details page with document timeline

• Document upload and processing status

• AI question-and-answer panel

• Source-reference panel

• Report review and approval screen

• Export/download screen

13. Recommended Folder Structure

```
clinical-workflow-platform/
■■■ app.py
■■■ config.py
■■■ requirements.txt
■■■ .env
■■■ README.md
■■■ routes/
■   ■■■ auth_routes.py
■   ■■■ patient_routes.py
■   ■■■ document_routes.py
■   ■■■ report_routes.py
■■■ services/
■   ■■■ document_processor.py
■   ■■■ embedding_service.py
■   ■■■ vector_store.py
■   ■■■ rag_service.py
■   ■■■ gemini_service.py
■■■ models/
■   ■■■ user_model.py
■   ■■■ patient_model.py
■   ■■■ document_model.py
■   ■■■ report_model.py
■■■ templates/
■   ■■■ login.html
■   ■■■ dashboard.html
■   ■■■ patients.html
■   ■■■ patient_detail.html
■■■ static/
■   ■■■ css/
■   ■■■ js/
■■■ uploads/
■■■ vector_store/
■■■ tests/
    ■■■ test_auth.py
    ■■■ test_documents.py
    ■■■ test_rag.py
```

14. Example Demo Scenario

Input: A user uploads a sample clinical report named `patient_report_01.pdf`.

Question: “Summarize the important findings from the previous report.”

System behavior: The application retrieves the most relevant sections, sends them with the question to Gemini, and displays a concise answer with the source filename and section references.

Expected result: The user can verify the answer against the uploaded document before approving or exporting it.

15. Safety and Reliability

• Use synthetic or de-identified data for the prototype.

• Display a clear notice that the system is a decision-support prototype, not a diagnostic tool.

• Do not allow the model to invent missing clinical facts.

• If the answer is not supported by retrieved context, return: “The uploaded documents do not contain enough information to answer this.”

• Restrict access to patient records using authentication and authorization.

• Validate file type, file size, and uploaded content.

• Log important actions for traceability.

• Require human review before exporting an approved report.

16. Review and Approval Workflow

```
AI Generates Answer
        |
        v
User Reviews Answer
        |
        v
User Edits if Required
        |
        v
User Approves
        |
        v
Approved Report
        |
        v
Export

Suggested statuses:
GENERATED -> UNDER_REVIEW -> APPROVED -> EXPORTED
```

17. Development Plan

Phase	Deliverable
Phase 1	Set up Flask project, environment variables, Git, and basic UI.
Phase 2	Implement authentication and SQL patient management.
Phase 3	Implement document upload, validation, and storage.
Phase 4	Implement text extraction and chunking.
Phase 5	Implement embeddings and FAISS indexing.
Phase 6	Integrate Gemini with grounded prompts.
Phase 7	Build source references, review, and export.
Phase 8	Add testing, error handling, documentation, and deployment.

18. CTO Explanation

> “I wanted to build a healthcare application that complements existing clinical AI systems rather than duplicating them. My project focuses on the information workflow around clinical documents. I used Flask for the backend, SQL for structured data, FAISS for semantic search, and Gemini for grounded question answering. The main challenge was making sure the AI answers were based on retrieved document context rather than generating unsupported information. I also implemented document processing, API validation, review workflows, and testing.”

19. Final Positioning

This project is intentionally designed as a supporting clinical-information platform. It demonstrates practical software engineering, AI integration, RAG, database design, API development, and reliability practices using the stated resume skills. It should be presented as an independent prototype inspired by healthcare workflow challenges—not as an existing BrainSightAI product and not as a replacement for VoxelBox or Snowdrop.

Final architecture in one line: Frontend → Flask API → SQL / Document Processing → FAISS Retrieval → Gemini → Source References → Human Review → Export.